

Atividade 08 – Sistema de Comunicação Segura entre Processos Distribuídos

Objetivo

Construir um sistema distribuído simples de troca de mensagens entre processos, aplicando os principais mecanismos de segurança estudados em aula: autenticação, criptografia e controle de acesso. O sistema deve ser composto por um servidor central e dois ou mais clientes conectados via rede.

Arquitetura

Nesta atividade, vocês devem garantir que na arquitetura:

- Apenas usuários cadastrados consigam se conectar (autenticação).
- As mensagens não possam ser lidas por terceiros (criptografia).
- Cada usuário só acesse o que tem permissão (controle de acesso).

O sistema é composto por três partes que rodam em processos separados:

Componente	Local	Tecnologia	Responsabilidade
Servidor Central	Notebook A	Python (Flask ou socket TCP)	Autentica clientes, armazena mensagens, controla quem pode ler o quê
Cliente A (Remetente)	Notebook B	Python (requests ou socket)	Faz login, criptografa a mensagem e a envia ao servidor
Cliente B (Destinatário)	Notebook B ou C	Python (requests ou socket)	Faz login e busca suas mensagens (já descriptografadas pelo servidor)

O que o sistema deve fazer

1. Autenticação de Usuários

O servidor deve manter uma lista de usuários cadastrados com login e senha. Nenhuma mensagem pode ser enviada ou recebida sem login prévio.

- A senha não pode ser armazenada em texto claro — use hash SHA-256 (biblioteca hashlib do Python).
- Ao fazer login, o servidor gera um token de sessão (string aleatória com uuid ou secrets) e o devolve ao cliente.
- Todas as requisições seguintes devem incluir esse token no cabeçalho.

2. Criptografia das Mensagens

As mensagens devem ser criptografadas antes de trafegar pela rede.

- Use criptografia simétrica com a biblioteca cryptography do Python (Fernet).
- A chave de criptografia deve ser a mesma para todos os usuários do sistema nesta versão simplificada.
- O servidor armazena as mensagens já criptografadas e as descriptografa apenas na entrega ao destinatário.

3. Controle de Acesso

Implemente regras simples de quem pode fazer o quê no sistema:

Ação	Papel: user	Papel: admin	Observação
Enviar mensagem	Sim	Sim	Para qualquer usuário cadastrado
Ler suas próprias mensagens	Sim	Sim	Apenas as mensagens destinadas a si
Ler mensagens de outros	Não	Sim	Somente admin enxerga todas as mensagens
Cadastrar novo usuário	Não	Sim	Via endpoint exclusivo do admin
Ver lista de usuários ativos	Não	Sim	Endpoint protegido por papel

4. Armazenamento

As mensagens devem ser persistidas no servidor entre sessões. Use SQLite com a estrutura sugerida:

Campo	Tipo	Descrição
id	INTEGER PK	Identificador automático da mensagem
remetente	TEXT	Username de quem enviou
destinatario	TEXT	Username do destinatário
conteudo_cifrado	BLOB	Mensagem criptografada com Fernet
timestamp	TEXT	Data/hora do envio (ISO 8601)
lida	INTEGER	0 = não lida, 1 = lida

Fluxo de apresentação

1. Servidor rodando e aguardando conexões.
 2. Tentativa de login com senha errada → acesso negado.
 3. Login com credenciais válidas → token gerado.
 4. Envio de mensagem criptografada de A para B.
 5. Leitura das mensagens por B (somente as dele).
 6. Tentativa de um usuário comum ler mensagens de outro → erro.
 7. Login como admin e leitura de todas as mensagens.
 8. Mostrar o conteúdo do banco SQLite direto — mensagens aparecem cifradas.
-

Entrega

Postar no SIGAA apenas o link do video e do repositório GitHub contendo:

- Código-fonte do servidor e dos clientes, organizados em pastas:
 - server/
 - client/
- README.md com:
 - Descrição do projeto e da arquitetura de segurança implementada.
 - Instruções de instalação (pip install -r requirements.txt).
 - Passo a passo para executar servidor e cliente.
 - Prints demonstrando o fluxo completo.
- requirements.txt listando todas as dependências.